

PDAs Accept Context-Free Languages

Theorem:

$$\left\{ \begin{array}{l} \text{Context-Free} \\ \text{Languages} \\ \text{(Grammars)} \end{array} \right\} = \left\{ \begin{array}{l} \text{Languages} \\ \text{Accepted by} \\ \text{PDAs} \end{array} \right\}$$

Proof - Step 1:

$$\left\{ \begin{array}{c} \text{Context-Free} \\ \text{Languages} \\ \text{(Grammars)} \end{array} \right\} \subseteq \left\{ \begin{array}{c} \text{Languages} \\ \text{Accepted by} \\ \text{PDAs} \end{array} \right\}$$

Convert any context-free grammar G
to a PDA M with: $L(G) = L(M)$

Proof - Step 2:

$$\left\{ \begin{array}{c} \text{Context-Free} \\ \text{Languages} \\ \text{(Grammars)} \end{array} \right\} \supseteq \left\{ \begin{array}{c} \text{Languages} \\ \text{Accepted by} \\ \text{PDAs} \end{array} \right\}$$

Convert any PDA M to a context-free grammar G with: $L(G) = L(M)$

Proof - step 1

Convert

Context-Free Grammars
to
PDAs

Take an arbitrary context-free grammar G

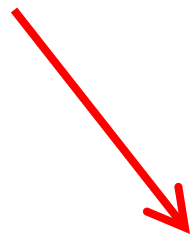
We will convert G to a PDA M such that:

$$L(G) = L(M)$$

Conversion Procedure:

For each
production in G

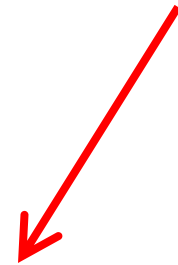
$$A \rightarrow w$$



$$\lambda, A \rightarrow w$$

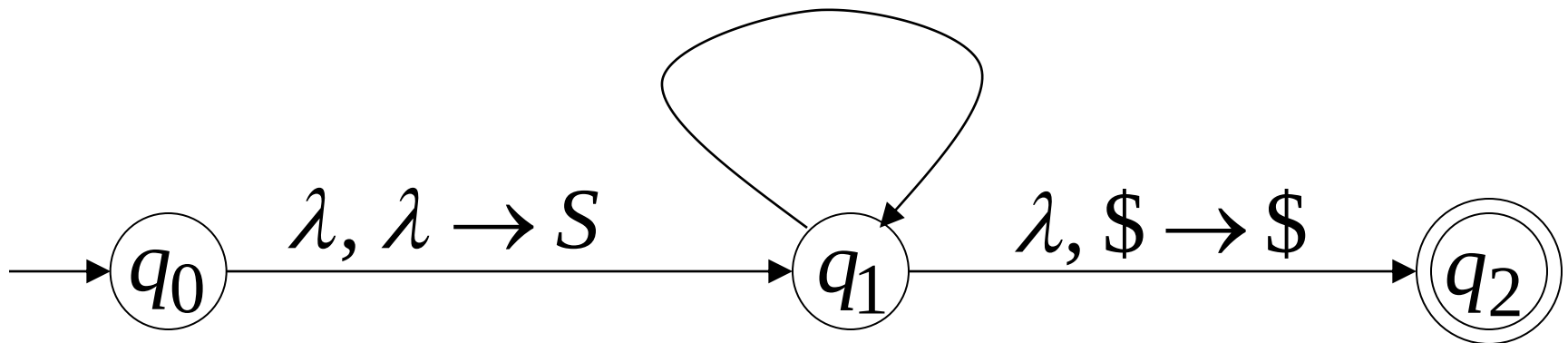
For each
terminal in G

a



$$a, a \rightarrow \lambda$$

Add transitions



Example

Grammar

$S \rightarrow aSTb$

$S \rightarrow b$

$T \rightarrow Ta$

$T \rightarrow \lambda$

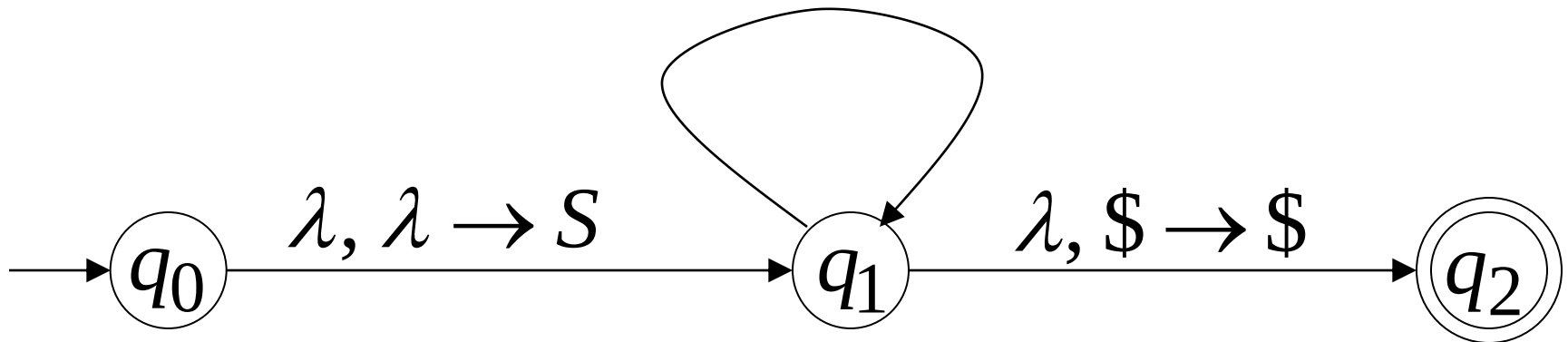
PDA

$\lambda, S \rightarrow aSTb$

$\lambda, S \rightarrow b$

$\lambda, T \rightarrow Ta \quad a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda \quad b, b \rightarrow \lambda$



PDA simulates leftmost derivations

Grammar

Leftmost Derivation

S

$\Rightarrow \dots$

$\Rightarrow \sigma_1 \cdots \sigma_k X_1 \cdots X_m$

$\Rightarrow \dots$

$\Rightarrow \sigma_1 \cdots \sigma_k \sigma_{k+1} \cdots \sigma_n$

PDA Computation

$(q_0, \sigma_1 \cdots \sigma_k \sigma_{k+1} \cdots \sigma_n, \$)$

$\succ (q_1, \sigma_1 \cdots \sigma_k \sigma_{k+1} \cdots \sigma_n, S\$)$

$\succ \dots$

$\succ (q_1, \sigma_{k+1} \cdots \sigma_n, X_1 \cdots X_m \$)$

$\succ \dots$

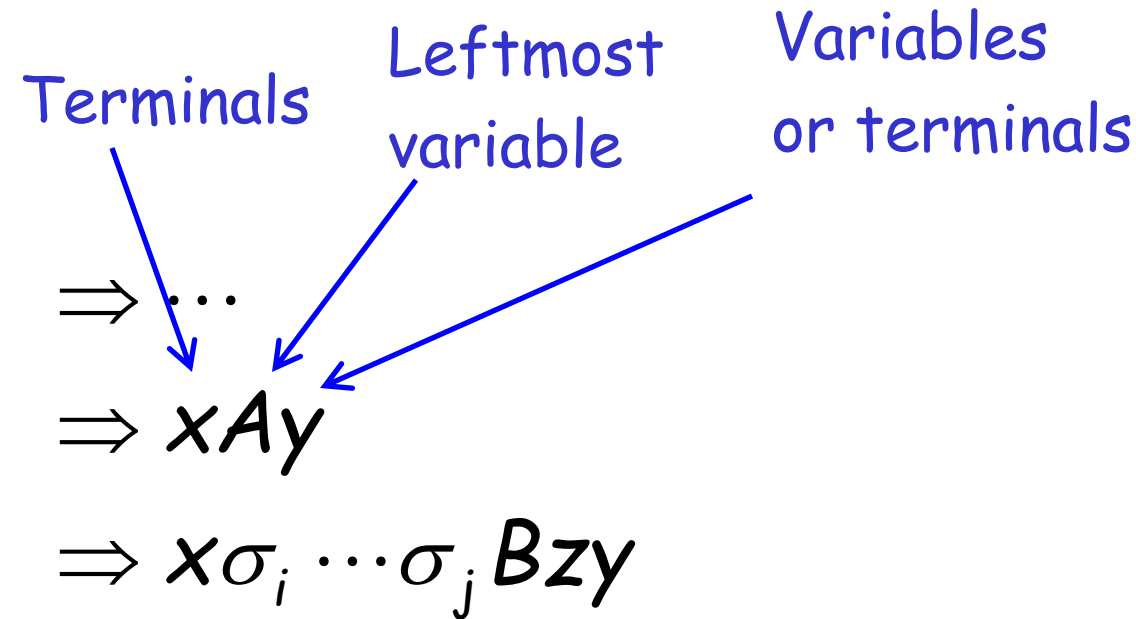
$\succ (q_2, \lambda, \$)$

Scanned
symbols

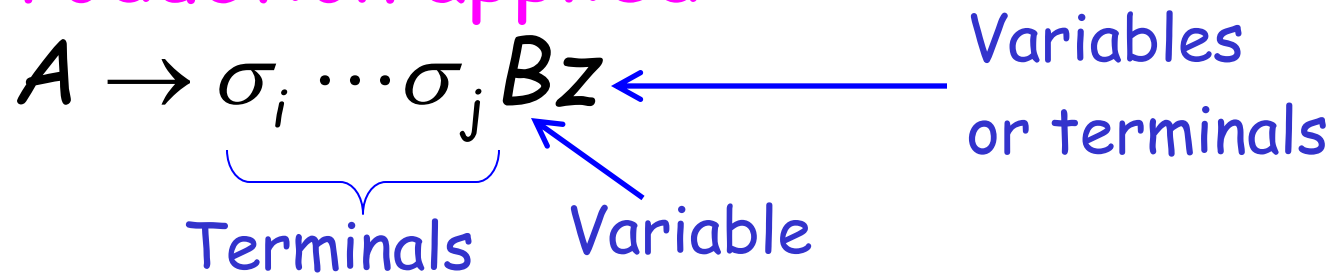
Stack
contents

Grammar

Leftmost Derivation



Production applied



Grammar

Leftmost Derivation

$\Rightarrow \dots$

$\Rightarrow xAy$

$\Rightarrow x\sigma_i \cdots \sigma_j Bzy$

Production applied

$A \rightarrow \sigma_i \cdots \sigma_j Bz$

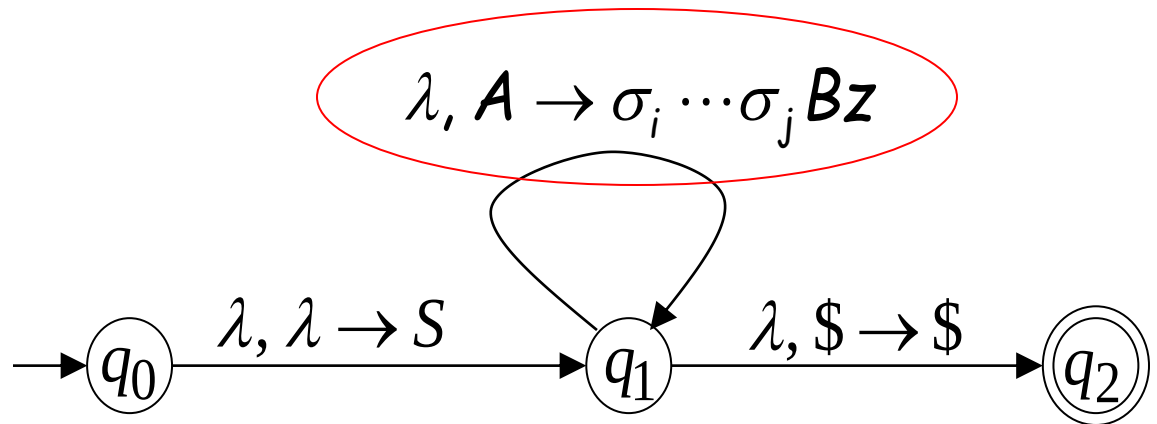
PDA Computation

$\succ \dots$

$\succ (q_1, \sigma_i \cdots \sigma_n, Ay\$)$

$\succ (q_1, \sigma_i \cdots \sigma_n, \sigma_i \cdots \sigma_j Bzy\$)$

Transition applied



Grammar

Leftmost Derivation

$\Rightarrow \dots$

$\Rightarrow xAy$

$\Rightarrow x\sigma_i \cdots \sigma_j Bzy$

PDA Computation

$\succ \dots$

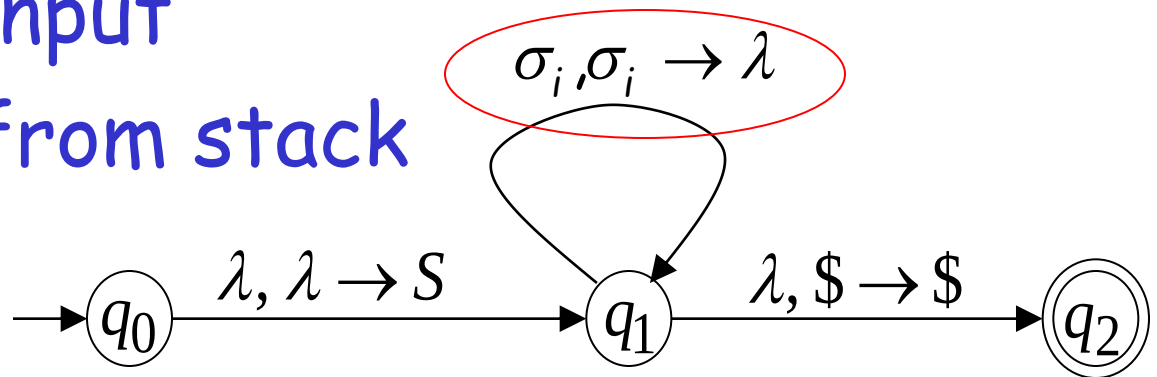
$\succ (q_1, \sigma_i \cdots \sigma_n, Ay\$)$

$\succ (q_1, \sigma_i \cdots \sigma_n, \sigma_i \cdots \sigma_j Bzy\$)$

$\succ (q_1, \sigma_{i+1} \cdots \sigma_n, \sigma_{i+1} \cdots \sigma_j Bzy\$)$

Read σ_i from input
and remove it from stack

Transition applied



Grammar

Leftmost Derivation

$\Rightarrow \dots$

$\Rightarrow xAy$

$\Rightarrow x\sigma_i \cdots \sigma_j Bzy$

PDA Computation

$\succ \dots$

$\succ (q_1, \sigma_i \cdots \sigma_n, Ay\$)$

$\succ (q_1, \sigma_i \cdots \sigma_n, \sigma_i \cdots \sigma_j Bzy\$)$

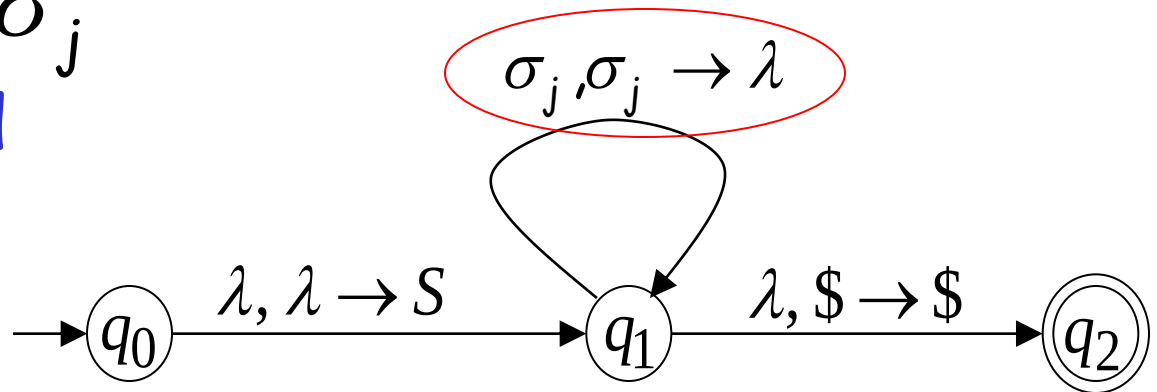
$\succ (q_1, \sigma_{i+1} \cdots \sigma_n, \sigma_{i+1} \cdots \sigma_j Bzy\$)$

$\succ \dots$

$\succ (q_1, \sigma_{j+1} \cdots \sigma_n, Bzy\$)$

Last Transition applied

All symbols $\sigma_i \cdots \sigma_j$
have been removed
from top of stack



The process repeats with the next
leftmost variable

$\Rightarrow \dots$

$\Rightarrow xAy$

$\Rightarrow x\sigma_i \cdots \sigma_j Bzy$

$\Rightarrow x\sigma_i \cdots \sigma_j \sigma_{j+1} \cdots \sigma_k Cpzy$

$\succ \dots$

$\succ (q_1, \sigma_{j+1} \cdots \sigma_n, Bzy \$)$

$\succ (q_1, \sigma_{j+1} \cdots \sigma_n, \sigma_{j+1} \cdots \sigma_k Cpzy \$)$

$\succ \dots$

$\succ (q_1, \sigma_{k+1} \cdots \sigma_n, Cpzy \$)$

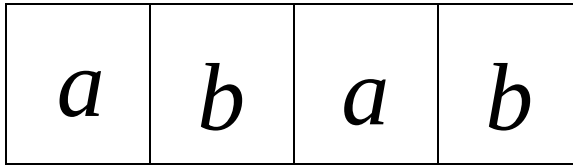
Production applied

$B \rightarrow \sigma_{j+1} \cdots \sigma_k Cp$

And so on.....

Example:

Input



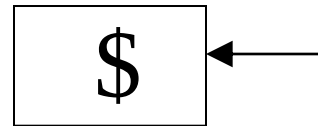
Time 0

$$\lambda, S \rightarrow aSTb$$

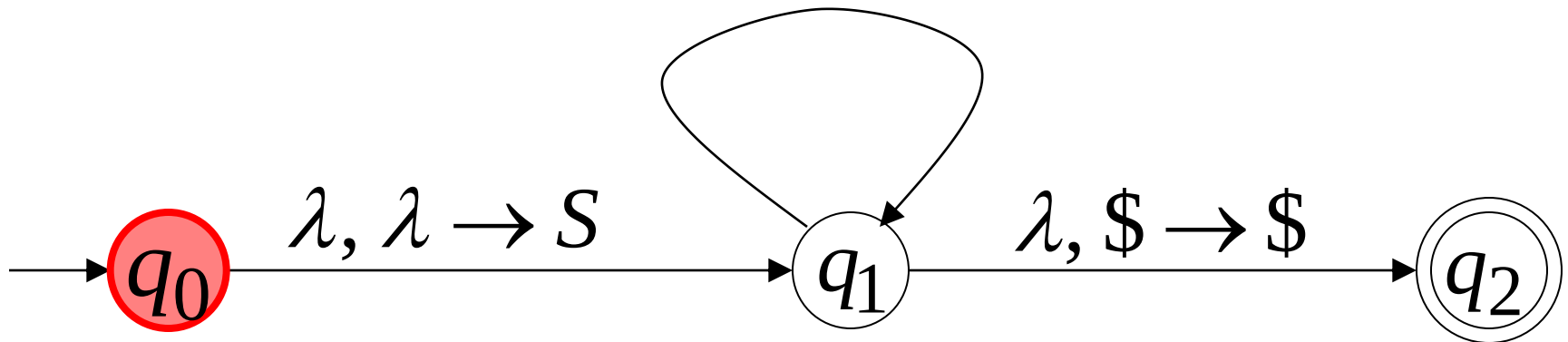
$$\lambda, S \rightarrow b$$

$$\lambda, T \rightarrow Ta \quad a, a \rightarrow \lambda$$

$$\lambda, T \rightarrow \lambda \quad b, b \rightarrow \lambda$$

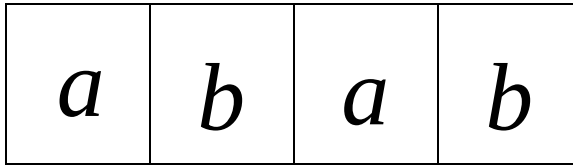


Stack



Derivation: S

Input



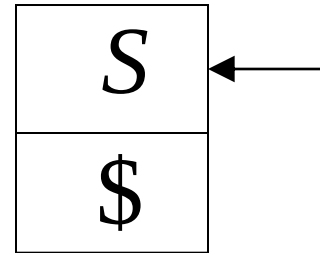
Time 1

$$\lambda, S \rightarrow aSTb$$

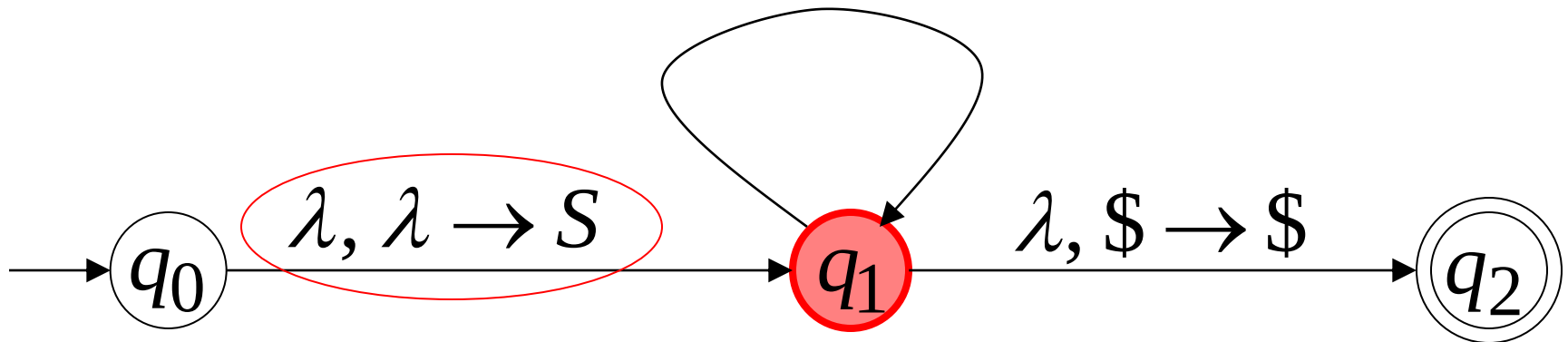
$$\lambda, S \rightarrow b$$

$$\lambda, T \rightarrow Ta \quad a, a \rightarrow \lambda$$

$$\lambda, T \rightarrow \lambda \quad b, b \rightarrow \lambda$$

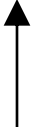
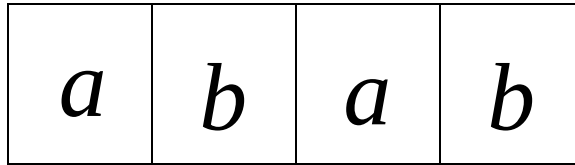


Stack



Derivation: $S \Rightarrow aSTb$

Input



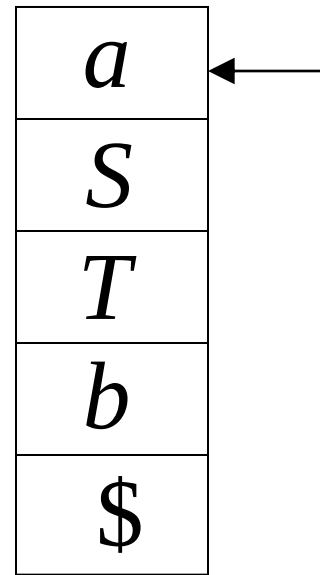
Time 2

$\lambda, S \rightarrow aSTb$

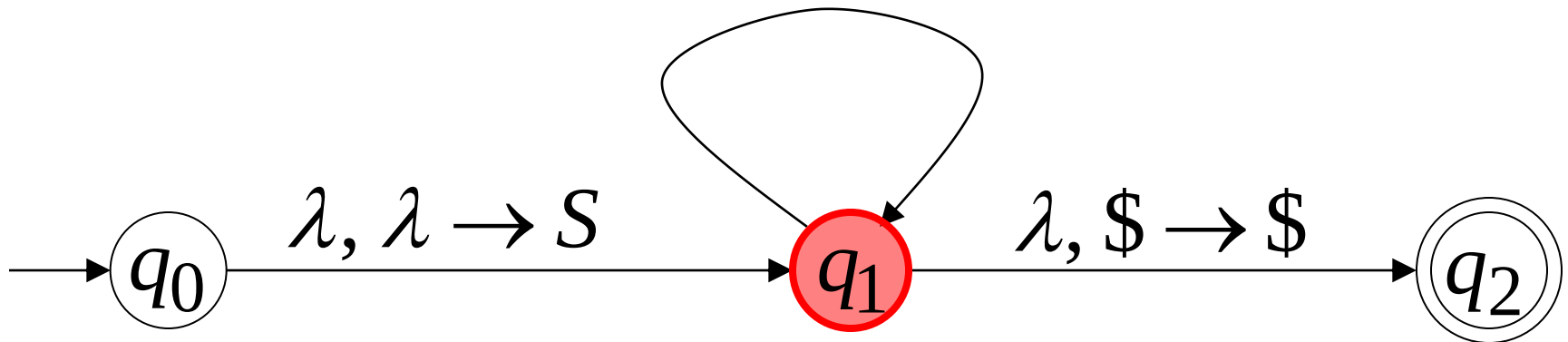
$\lambda, S \rightarrow b$

$\lambda, T \rightarrow Ta$ $a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda$ $b, b \rightarrow \lambda$

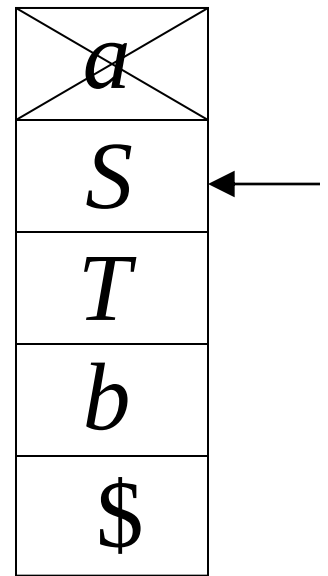
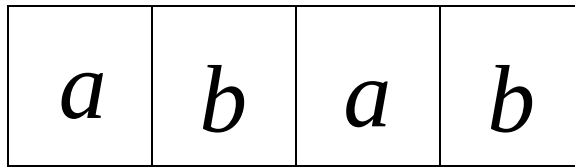


Stack



Derivation: $S \Rightarrow aSTb$

Input



Stack

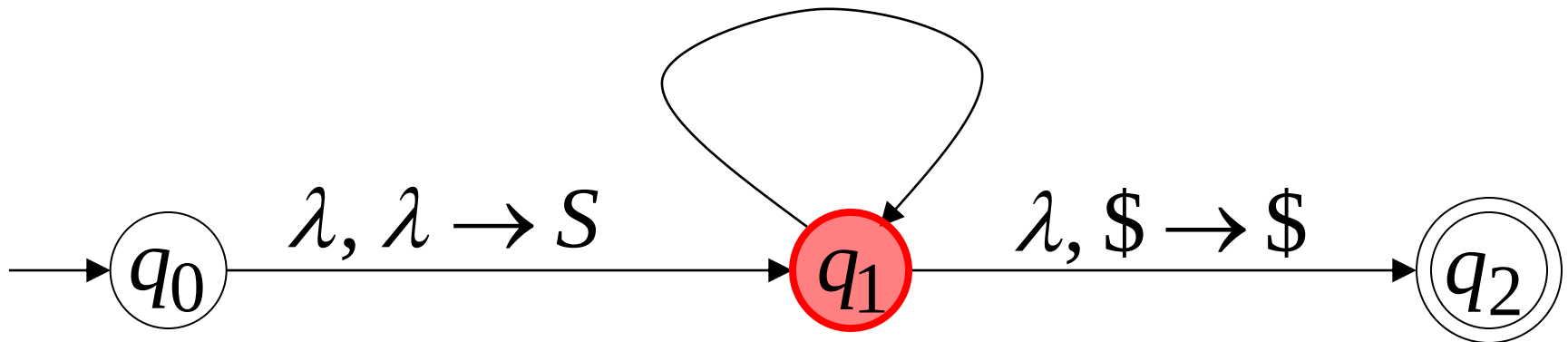
Time 3

$\lambda, S \rightarrow aSTb$

$\lambda, S \rightarrow b$

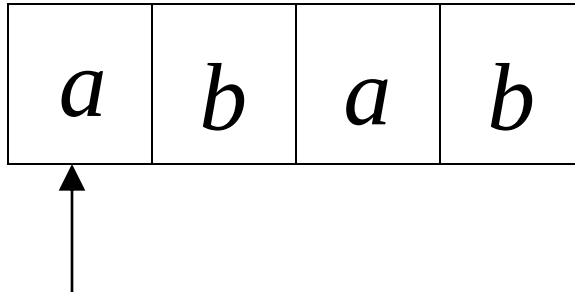
$\lambda, T \rightarrow Ta$ $a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda$ $b, b \rightarrow \lambda$



Derivation: $S \Rightarrow aSTb \Rightarrow abTb$

Input



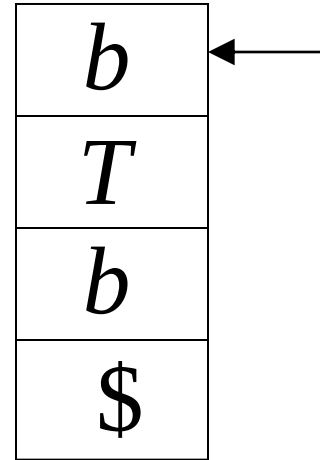
Time 4

$\lambda, S \rightarrow aSTb$

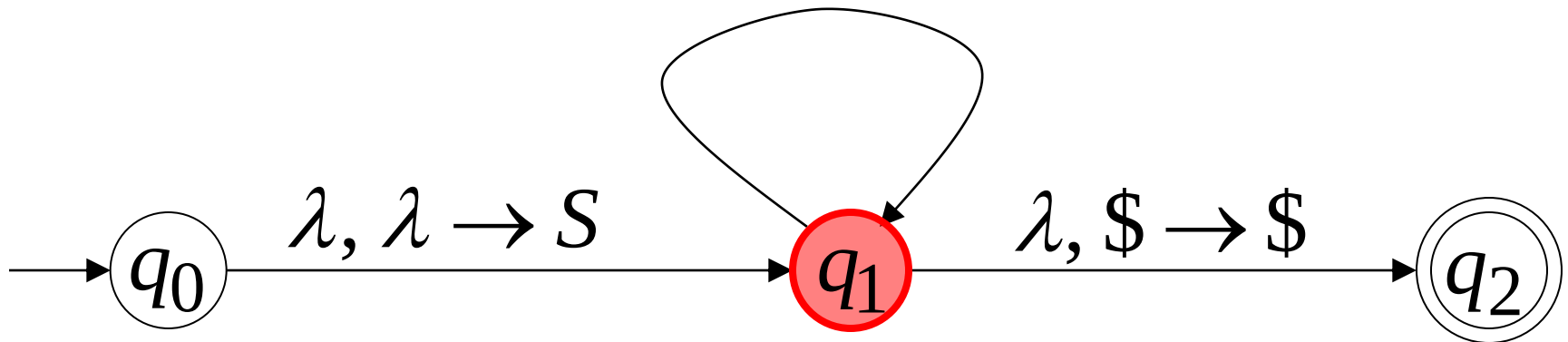
$\lambda, S \rightarrow b$

$\lambda, T \rightarrow Ta \quad a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda \quad b, b \rightarrow \lambda$

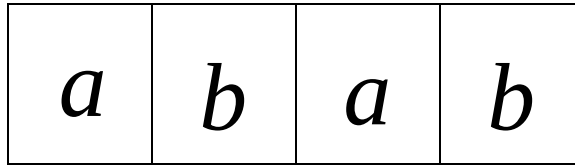


Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb$

Input



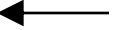
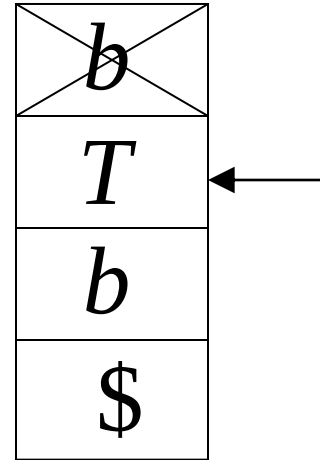
$\lambda, S \rightarrow aSTb$

Time 5

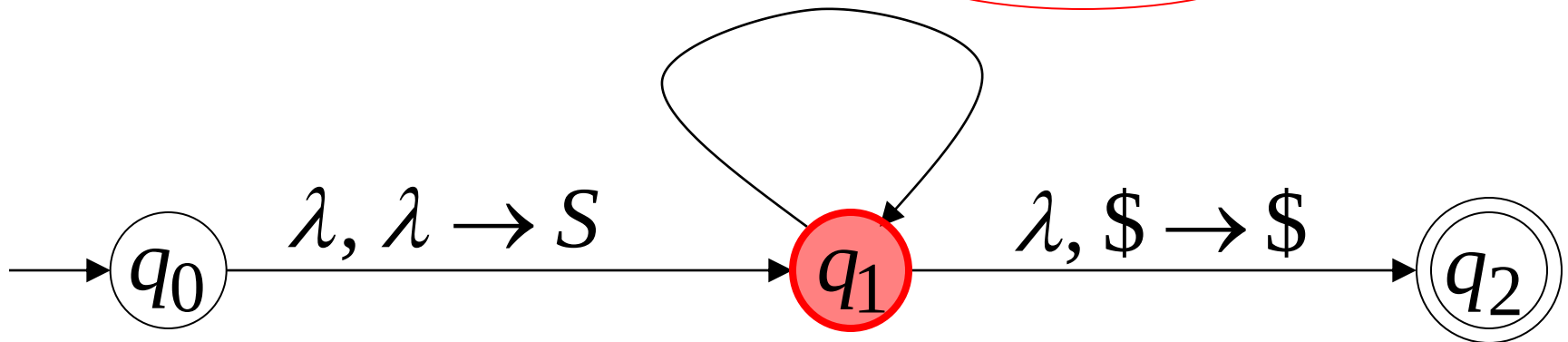
$\lambda, S \rightarrow b$

$\lambda, T \rightarrow Ta$ $a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda$ $b, b \rightarrow \lambda$

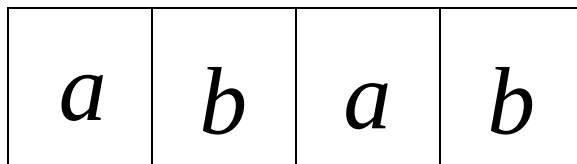


Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab$

Input

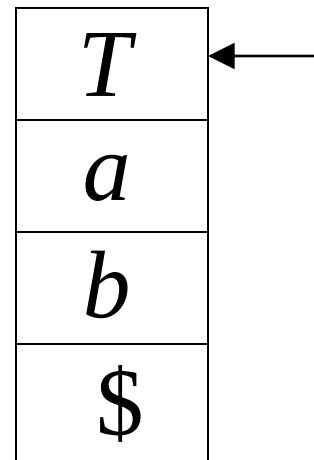


$\lambda, S \rightarrow aSTb$

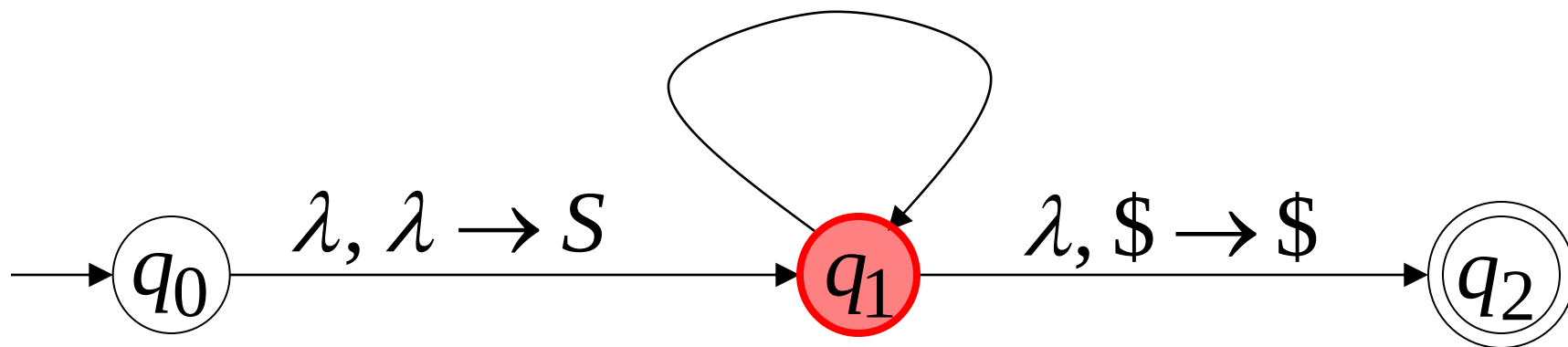
$\lambda, S \rightarrow b$

$\lambda, T \rightarrow Ta$ $a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda$ $b, b \rightarrow \lambda$

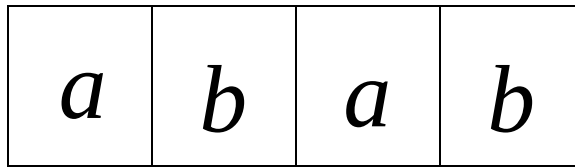


Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$

Input

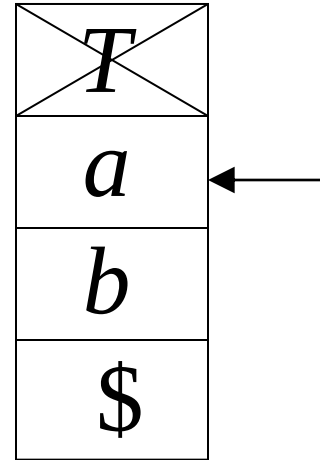


$\lambda, S \rightarrow aSTb$

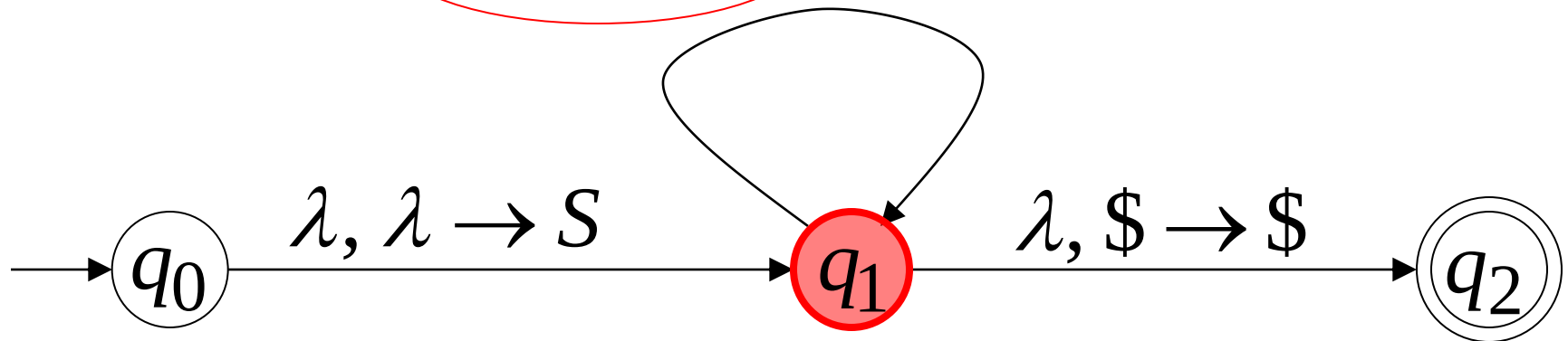
$\lambda, S \rightarrow b$

$\lambda, T \rightarrow Ta$ $a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda$ $b, b \rightarrow \lambda$

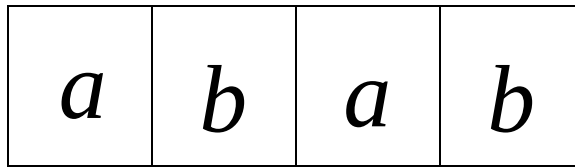


Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$

Input



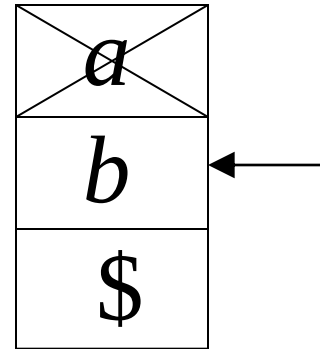
Time 8

$\lambda, S \rightarrow aSTb$

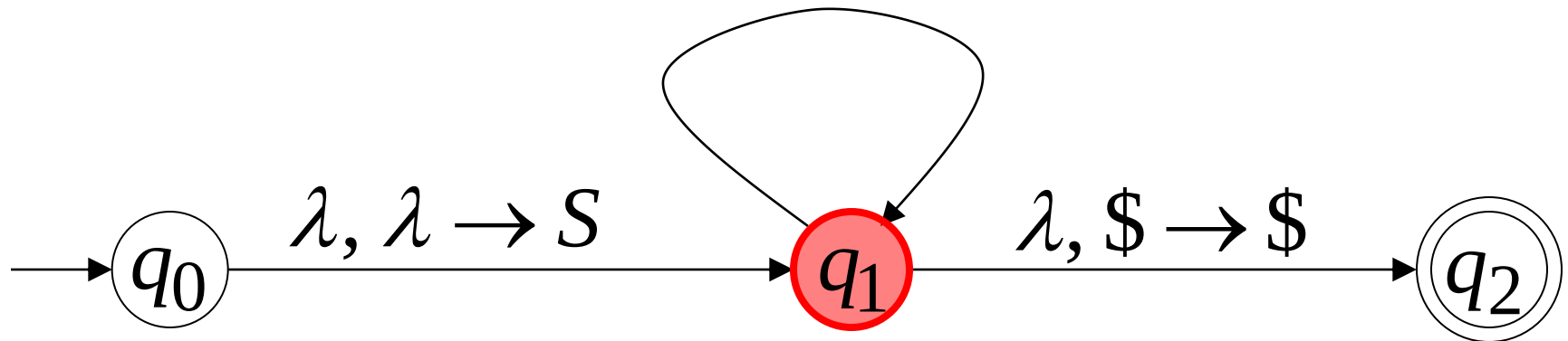
$\lambda, S \rightarrow b$

$\lambda, T \rightarrow Ta$ $a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda$ $b, b \rightarrow \lambda$

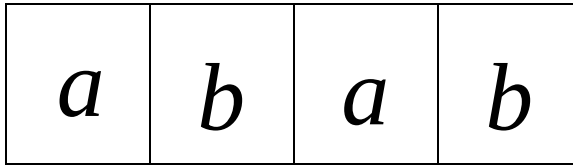


Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$

Input



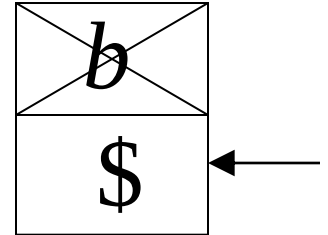
Time 9

$\lambda, S \rightarrow aSTb$

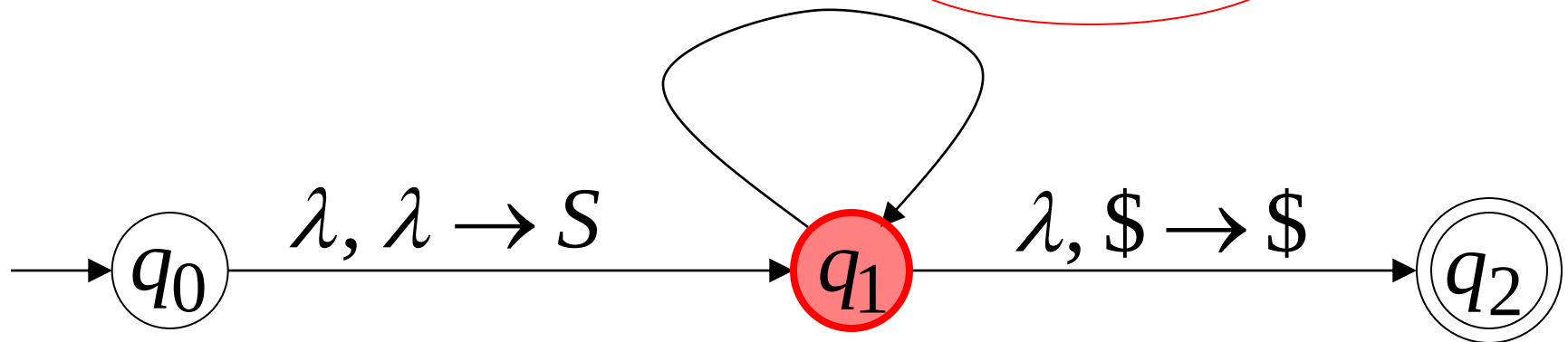
$\lambda, S \rightarrow b$

$\lambda, T \rightarrow Ta$ $a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda$ $b, b \rightarrow \lambda$

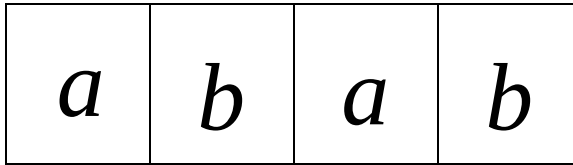


Stack



Derivation: $S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$

Input



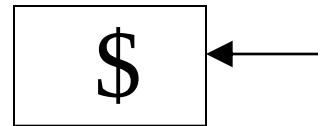
Time 10

$\lambda, S \rightarrow aSTb$

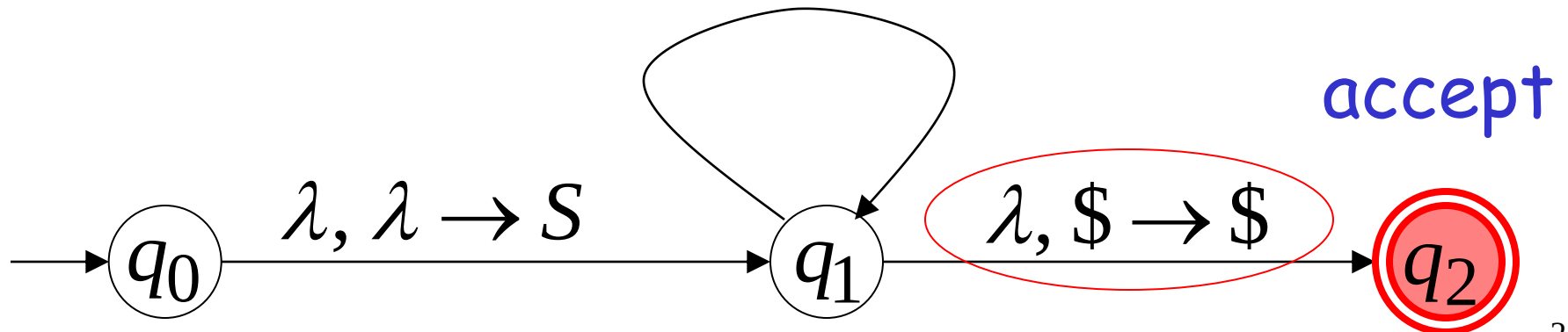
$\lambda, S \rightarrow b$

$\lambda, T \rightarrow Ta \quad a, a \rightarrow \lambda$

$\lambda, T \rightarrow \lambda \quad b, b \rightarrow \lambda$



Stack



Grammar

Leftmost Derivation

S

$\Rightarrow aSTb$

$\Rightarrow abTb$

$\Rightarrow abTab$

$\Rightarrow abab$

PDA Computation

$(q_0, abab, \$)$

$\succ (q_1, abab, S\$)$

$\succ (q_1, bab, STb\$)$

$\succ (q_1, bab, bTb\$)$

$\succ (q_1, ab, Tb\$)$

$\succ (q_1, ab, Tab\$)$

$\succ (q_1, ab, ab\$)$

$\succ (q_1, b, b\$)$

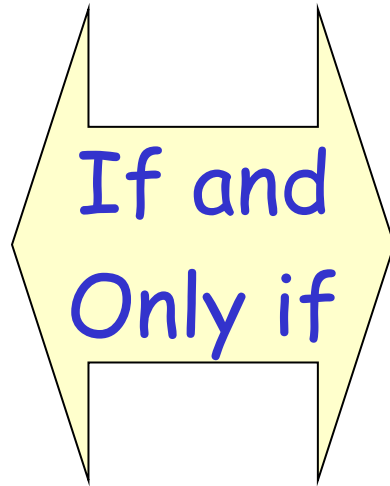
$\succ (q_1, \lambda, \$)$

$\succ (q_2, \lambda, \$)$

In general, it can be shown that:

Grammar G
generates
string w

$S \xRightarrow{*} w$



PDA M
accepts w

$(q_0, w, \$) \succ (q_2, \lambda, \$)$

Therefore $L(G) = L(M)$

Proof - step 2

Convert

PDAs

to

Context-Free Grammars

Take an arbitrary PDA M

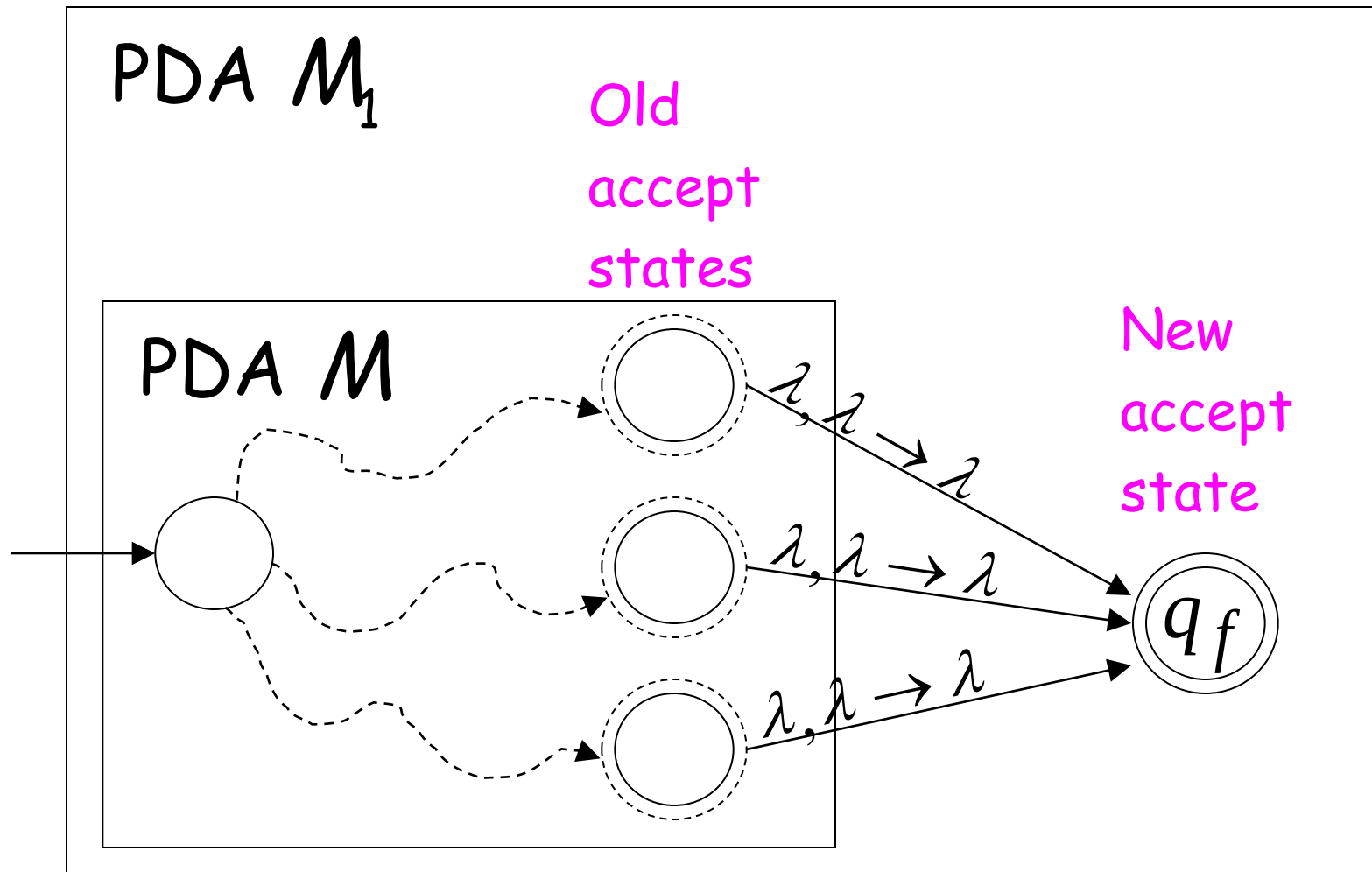
We will convert M
to a context-free grammar G such that:

$$L(M) = L(G)$$

First modify PDA M so that:

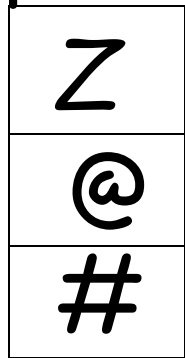
1. The PDA has a single accept state
2. Use new initial stack symbol $\#$
3. On acceptance the stack contains only stack symbol $\#$
4. Each transition either pushes a symbol or pops a symbol but not both together

1. The PDA has a single accept state



2. Use new initial stack symbol

Top of stack



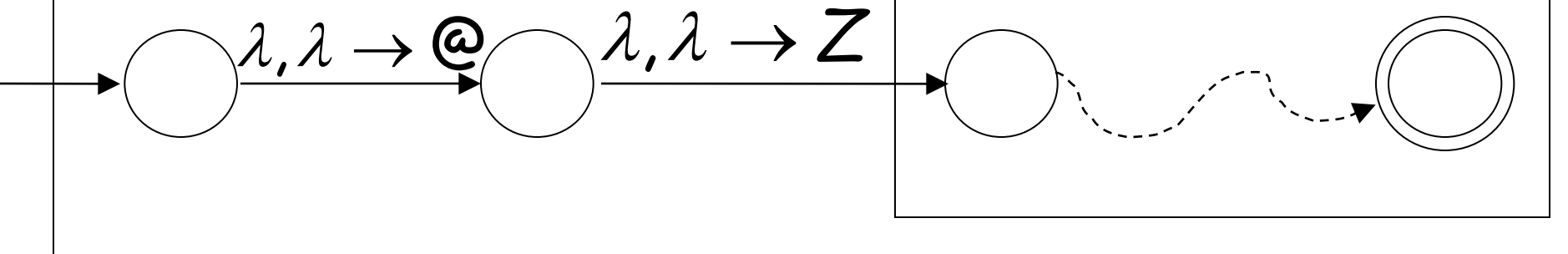
← old initial stack symbol

← auxiliary stack symbol

← new initial stack symbol

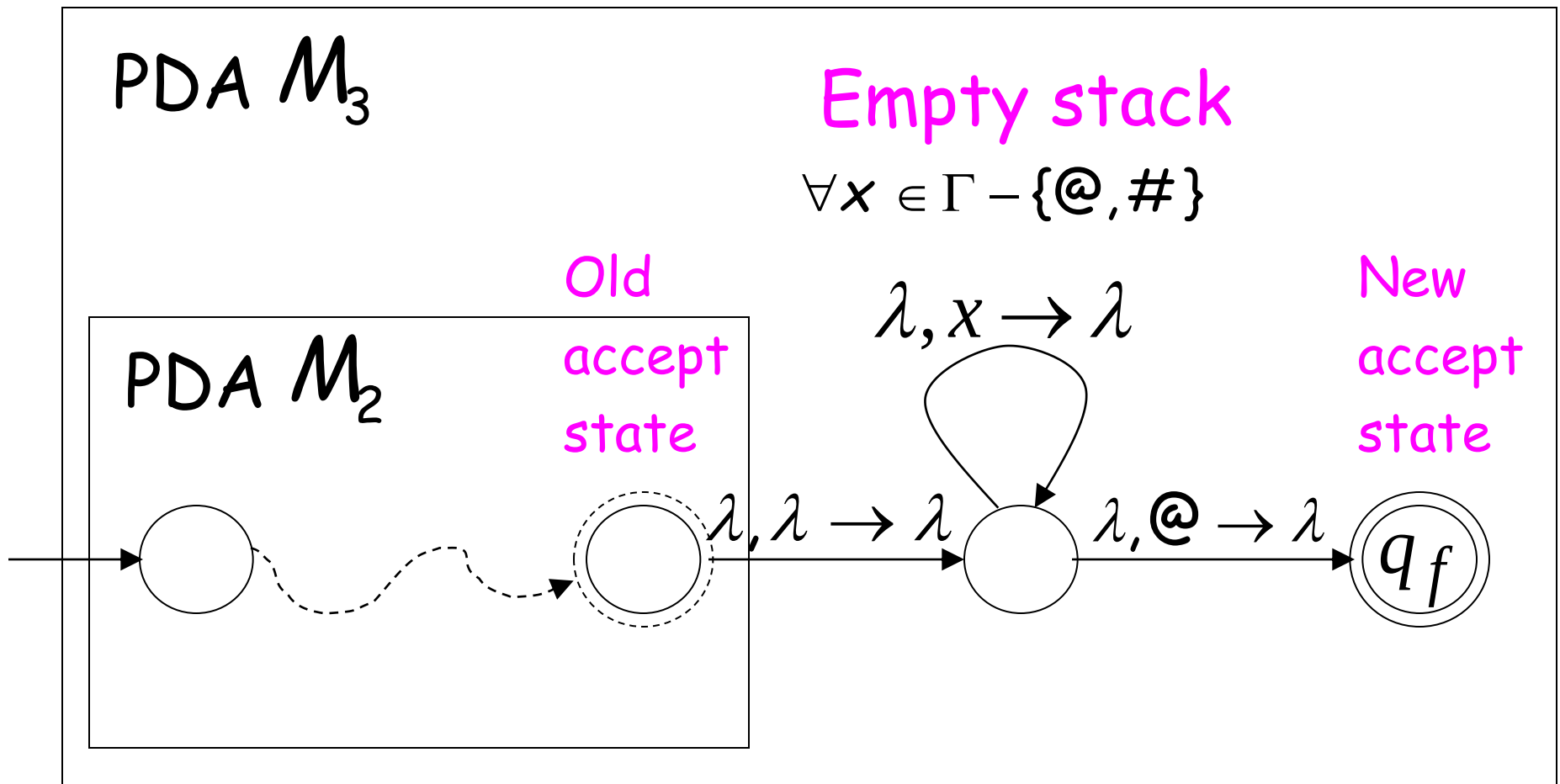
PDA M_2

PDA M_1

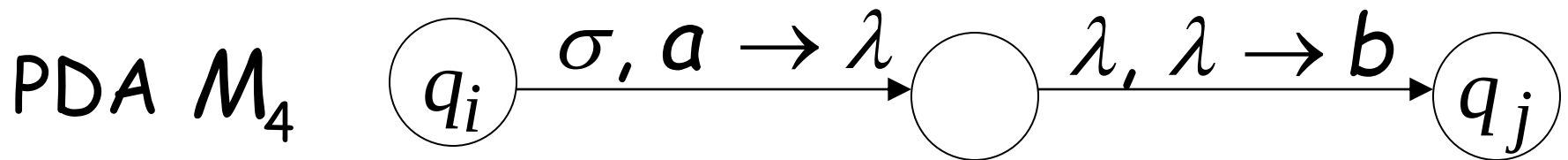
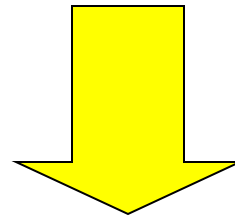
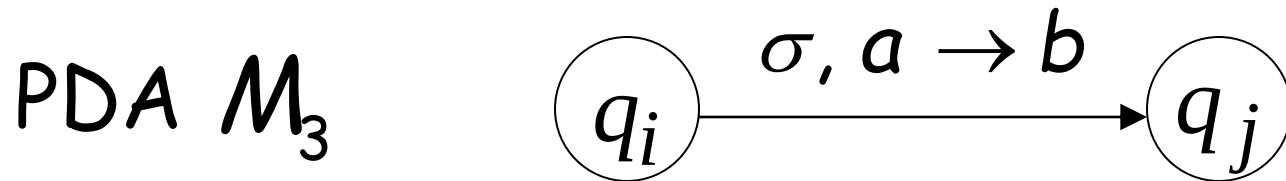


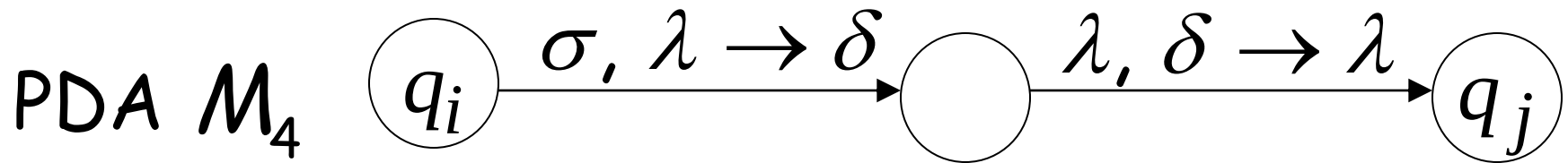
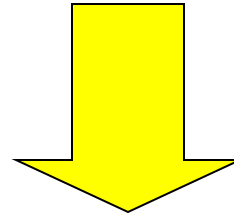
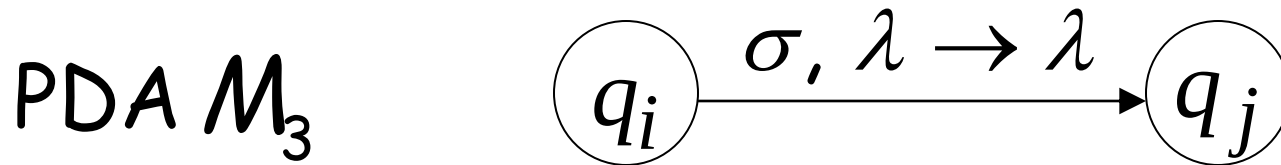
M_1 still thinks that Z is the initial stack

3. On acceptance the stack contains only stack symbol



4. Each transition either pushes a symbol or pops a symbol but not both together



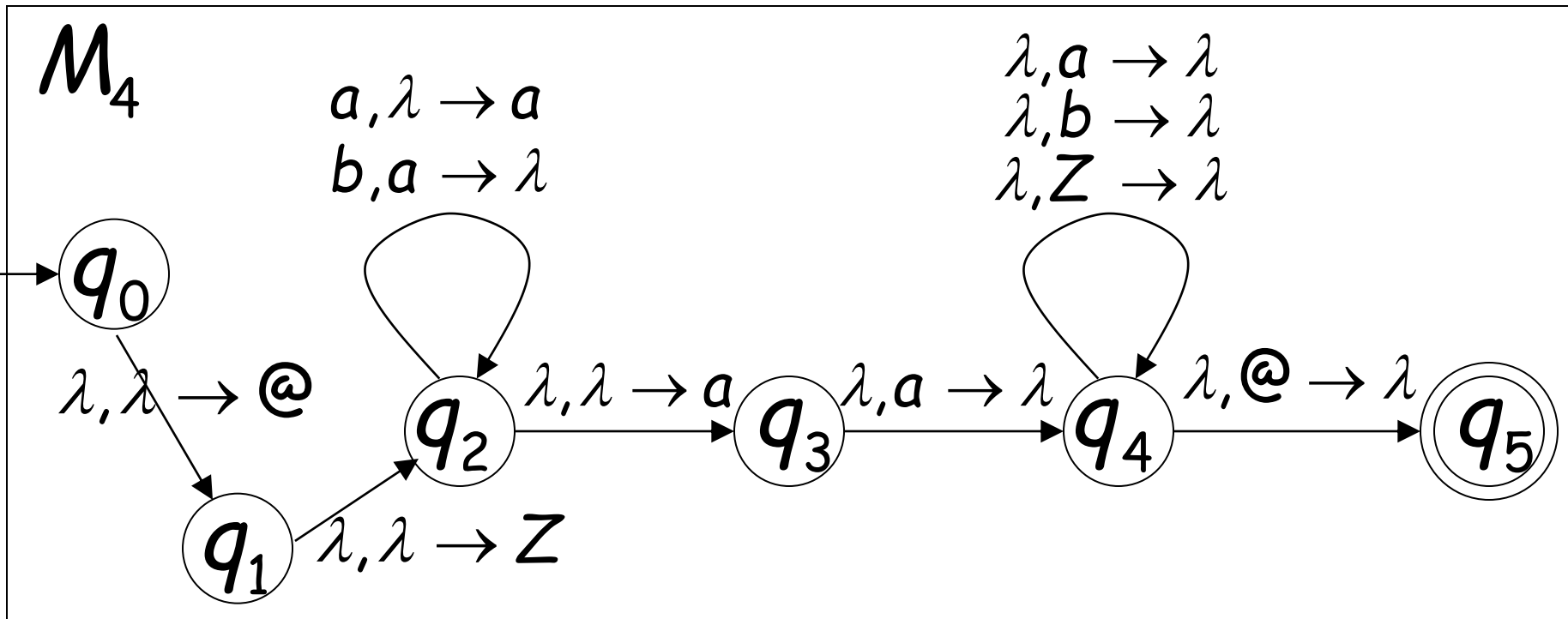
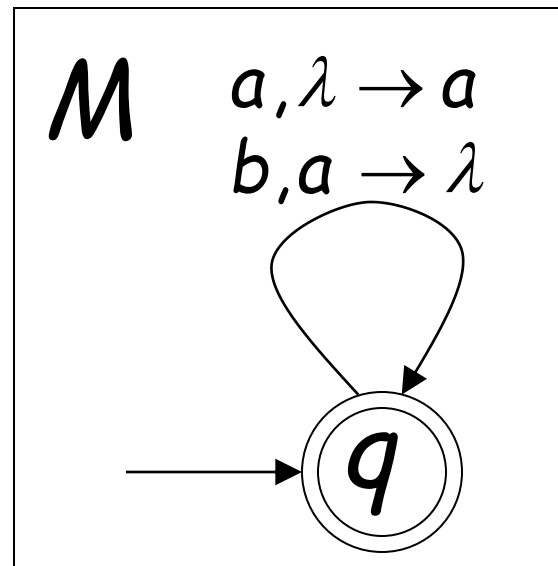


Where δ is a symbol of the stack alphabet

PDA M_4 is the final modified PDA

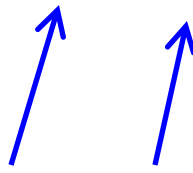
Note that the new initial stack symbol $\#$ is never used in any transition

Example:



Grammar Construction

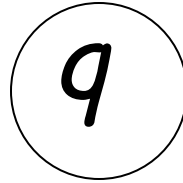
Variables: A_{q_i, q_j}



States of PDA

PDA

Kind 1: for each state

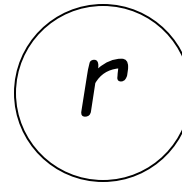
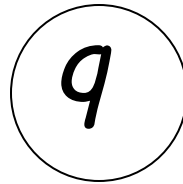
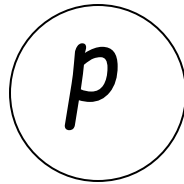


Grammar

$$A_{qq} \rightarrow \lambda$$

PDA

Kind 2: for every three states

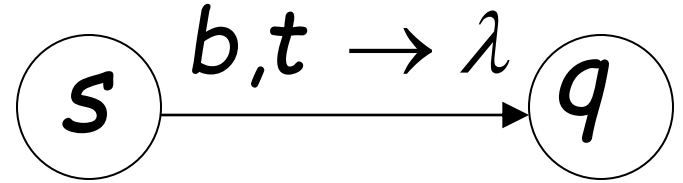
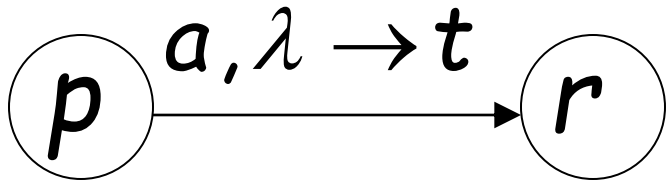


Grammar

$$A_{pq} \rightarrow A_{pr} A_{rq}$$

PDA

Kind 3: for every pair of such transitions

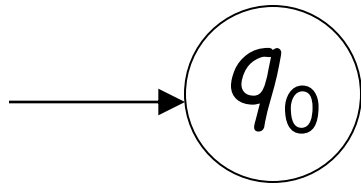


Grammar

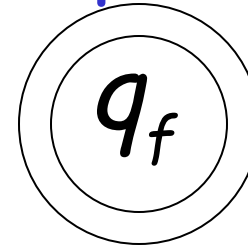
$$A_{pq} \rightarrow aA_{rs}b$$

PDA

Initial state



Accept state



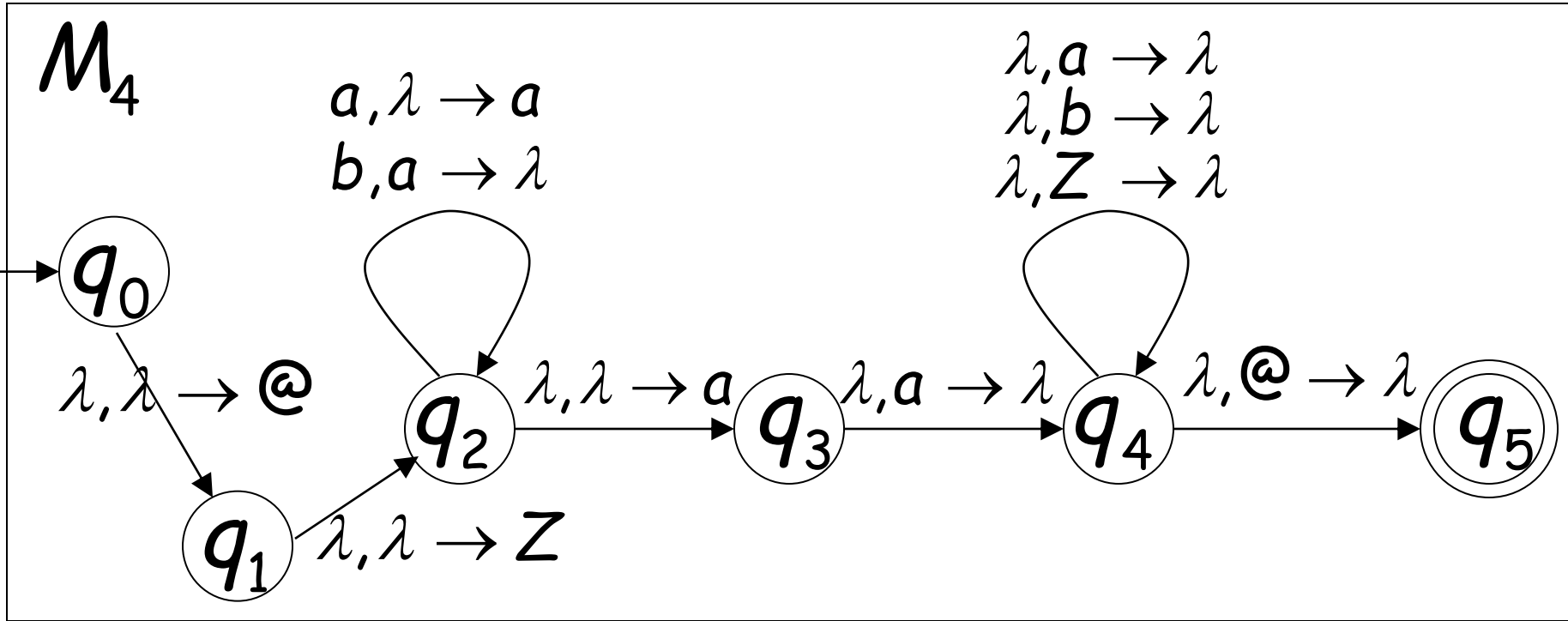
Grammar

Start variable

$A_{q_0 q_f}$

Example:

PDA



Grammar

Kind 1: from single states

$$A_{q_0q_0} \rightarrow \lambda$$

$$A_{q_1q_1} \rightarrow \lambda$$

$$A_{q_2q_2} \rightarrow \lambda$$

$$A_{q_3q_3} \rightarrow \lambda$$

$$A_{q_4q_4} \rightarrow \lambda$$

$$A_{q_5q_5} \rightarrow \lambda$$

Kind 2: from triplets of states

$$A_{q_0q_0} \rightarrow A_{q_0q_0} A_{q_0q_0} \mid A_{q_0q_1} A_{q_1q_0} \mid A_{q_0q_2} A_{q_2q_0} \mid A_{q_0q_3} A_{q_3q_0} \mid A_{q_0q_4} A_{q_4q_0} \mid A_{q_0q_5} A_{q_5q_0}$$

$$A_{q_0q_1} \rightarrow A_{q_0q_0} A_{q_0q_1} \mid A_{q_0q_1} A_{q_1q_1} \mid A_{q_0q_2} A_{q_2q_1} \mid A_{q_0q_3} A_{q_3q_1} \mid A_{q_0q_4} A_{q_4q_1} \mid A_{q_0q_5} A_{q_5q_1}$$

⋮

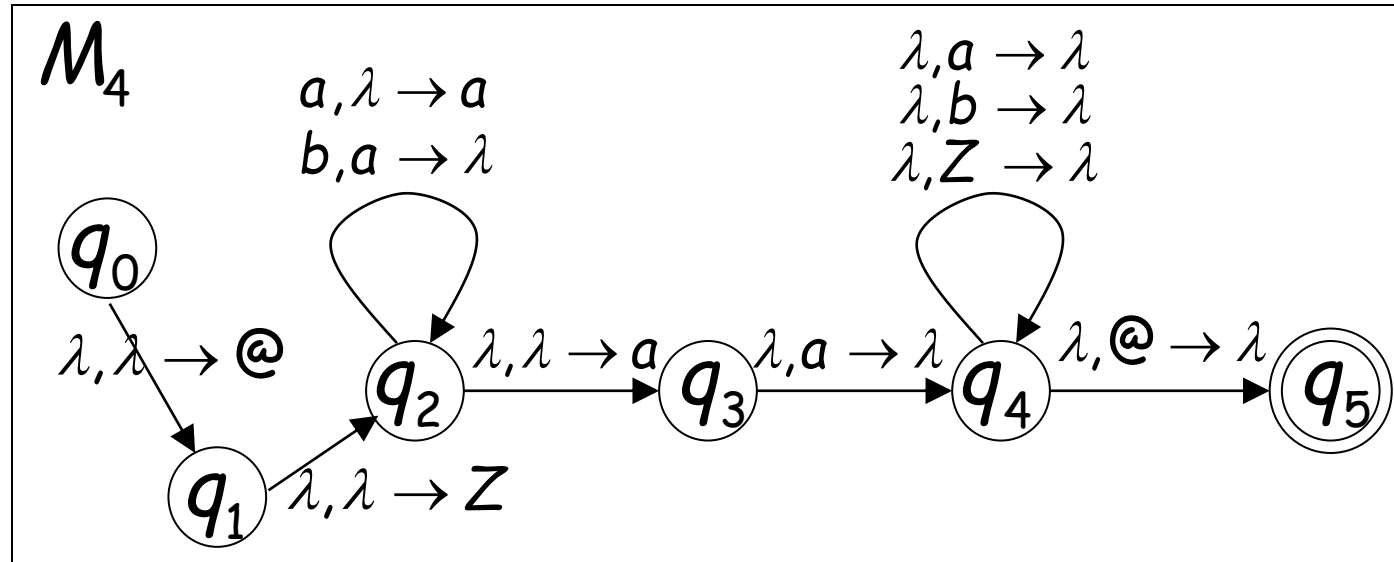
$$A_{q_0q_5} \rightarrow A_{q_0q_0} A_{q_0q_5} \mid A_{q_0q_1} A_{q_1q_5} \mid A_{q_0q_2} A_{q_2q_5} \mid A_{q_0q_3} A_{q_3q_5} \mid A_{q_0q_4} A_{q_4q_5} \mid A_{q_0q_5} A_{q_5q_5}$$

⋮

$$A_{q_5q_5} \rightarrow A_{q_5q_0} A_{q_0q_5} \mid A_{q_5q_1} A_{q_1q_5} \mid A_{q_5q_2} A_{q_2q_5} \mid A_{q_5q_3} A_{q_3q_5} \mid A_{q_5q_4} A_{q_4q_5} \mid A_{q_5q_5} A_{q_5q_5}$$

Start variable $A_{q_0q_5}$

Kind 3: from pairs of transitions



$$A_{q_0 q_5} \rightarrow A_{q_1 q_4}$$

$$A_{q_2 q_4} \rightarrow a A_{q_2 q_4}$$

$$A_{q_2 q_2} \rightarrow A_{q_3 q_2} b$$

$$A_{q_1 q_4} \rightarrow A_{q_2 q_4}$$

$$A_{q_2 q_2} \rightarrow a A_{q_2 q_2} b$$

$$A_{q_2 q_4} \rightarrow A_{q_3 q_3}$$

$$A_{q_2 q_4} \rightarrow a A_{q_2 q_3}$$

$$A_{q_2 q_4} \rightarrow A_{q_3 q_4}$$

Suppose that a PDA M is converted
to a context-free grammar G

We need to prove that $L(G) = L(M)$

or equivalently

$$L(G) \subseteq L(M)$$

$$L(G) \supseteq L(M)$$

$$L(G) \subseteq L(M)$$

We need to show that if G has derivation:

$$A_{q_0 q_f} \xRightarrow{*} w \quad (\text{string of terminals})$$

Then there is an accepting computation in M :

$$(q_0, w, \#) \xrightarrow{*} (q_f, \lambda, \#)$$

with input string w

We will actually show that if G has derivation:

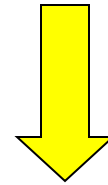
$$A_{pq} \stackrel{*}{\Rightarrow} w$$

Then there is a computation in M :

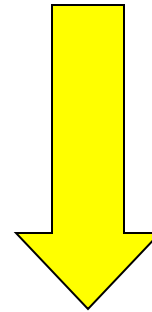
$$(p, w, \lambda) \stackrel{*}{\succ} (q, \lambda, \lambda)$$

Therefore:

$$A_{q_0 q_f} \stackrel{*}{\Rightarrow} w$$



$$(q_0, w, \lambda) \stackrel{*}{\succ} (q_f, \lambda, \lambda)$$



Since there is no transition
with the # symbol

$$(q_0, w, \#) \stackrel{*}{\succ} (q_f, \lambda, \#)$$

Lemma:

If $A_{pq} \xRightarrow{*} w$ (string of terminals)

then there is a computation
from state p to state q on string w
which leaves the stack empty:

$$(p, w, \lambda) \xRightarrow{*} (q, \lambda, \lambda)$$

Proof Intuition:

$$A_{pq} \Rightarrow \dots \Rightarrow W$$

Type 2

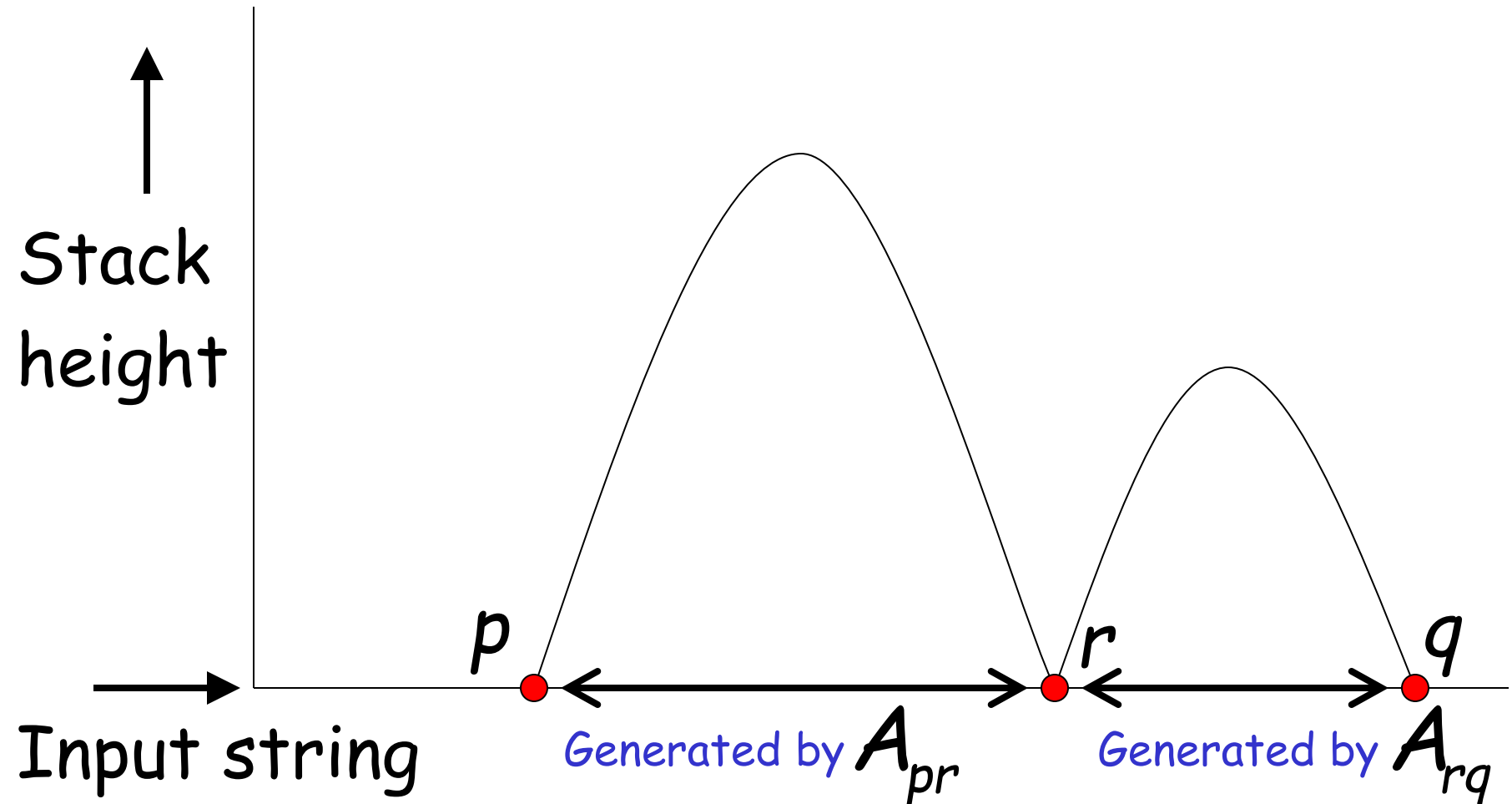
Case 1: $A_{pq} \Rightarrow A_{pr} A_{rq} \Rightarrow \dots \Rightarrow W$

Type 3

Case 2: $A_{pq} \Rightarrow a A_{rs} b \Rightarrow \dots \Rightarrow W$

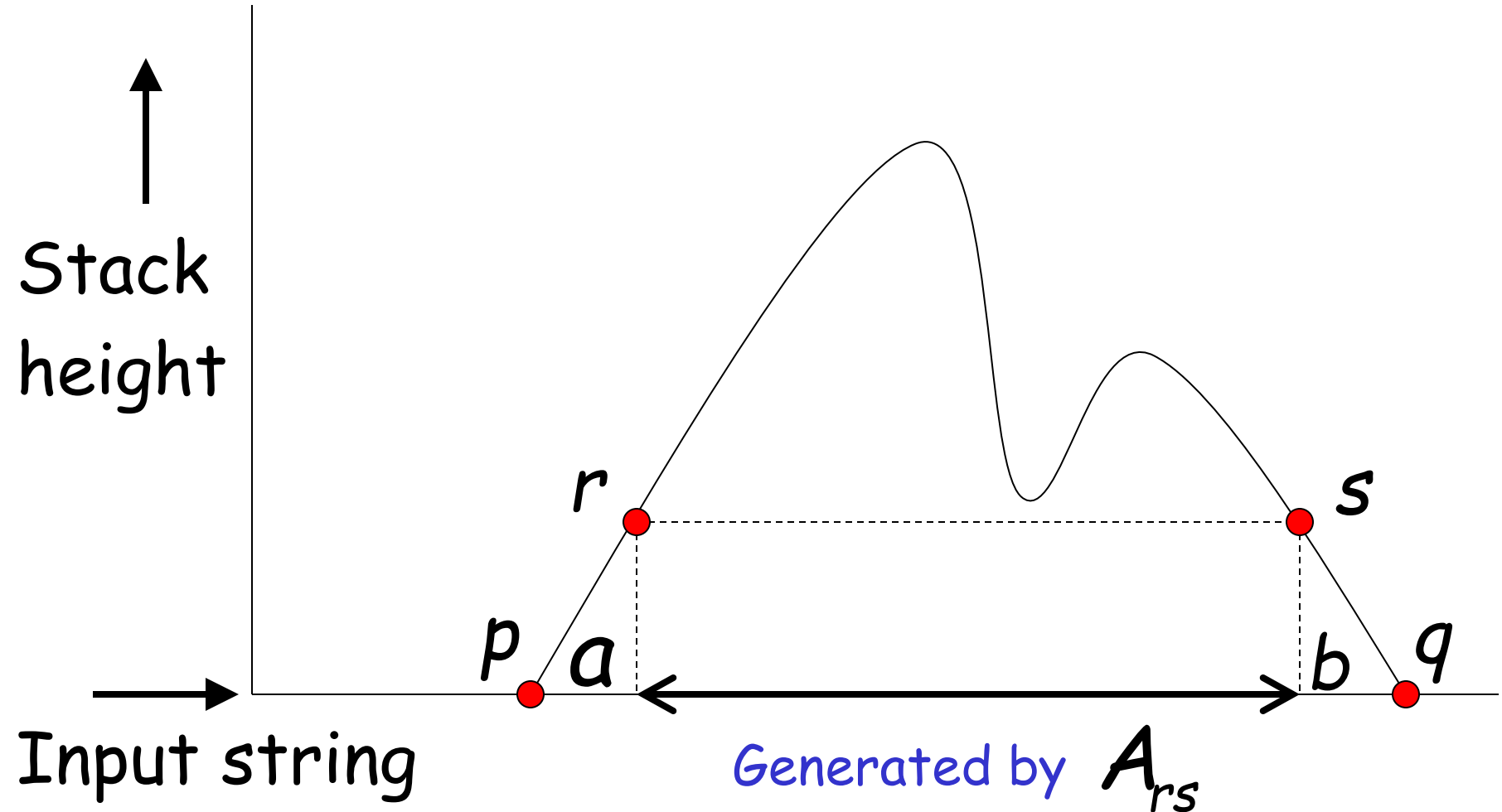
Type 2

Case 1: $A_{pq} \Rightarrow A_{pr} A_{rq} \Rightarrow \dots \Rightarrow w$



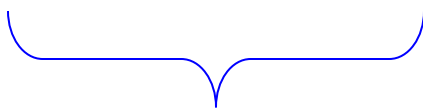
Type 3

Case 2: $A_{pq} \Rightarrow aA_{rs}b \Rightarrow \dots \Rightarrow w$



Formal Proof:

We formally prove this claim
by induction on the number
of steps in derivation:

$$A_{pq} \Rightarrow \cdots \Rightarrow W$$


number of steps

Induction Basis: $A_{pq} \Rightarrow w$

(one derivation step)

A Kind 1 production must have been used:

$$A_{pp} \rightarrow \lambda$$

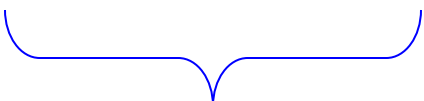
Therefore, $p = q$ and $w = \lambda$

This computation of PDA trivially exists:

$$(p, \lambda, \lambda) \stackrel{*}{\succ} (p, \lambda, \lambda)$$

Induction Hypothesis:

$$A_{pq} \Rightarrow \cdots \Rightarrow w$$

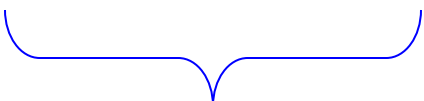

 k derivation steps

suppose it holds:

$$(p, w, \lambda) \stackrel{*}{\succ} (q, \lambda, \lambda)$$

Induction Step:

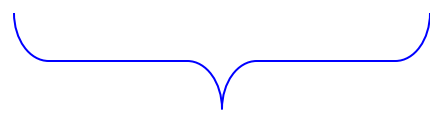
$$A_{pq} \Rightarrow \cdots \Rightarrow w$$


 $k + 1$ derivation steps

We have to show:

$$(p, w, \lambda) \stackrel{*}{\succ} (q, \lambda, \lambda)$$

$$A_{pq} \Rightarrow \dots \Rightarrow w$$



$k + 1$ derivation steps

Type 2

Case 1: $A_{pq} \Rightarrow A_{pr} A_{rq} \Rightarrow \dots \Rightarrow w$

Type 3

Case 2: $A_{pq} \Rightarrow a A_{rs} b \Rightarrow \dots \Rightarrow w$

Type 2

Case 1: $A_{pq} \Rightarrow A_{pr} A_{rq} \Rightarrow \dots \Rightarrow w$

$k + 1$ steps

We can write $w = yz$

$$A_{pr} \Rightarrow \dots \Rightarrow y$$

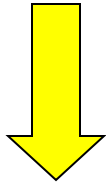
At most k steps

$$A_{rq} \Rightarrow \dots \Rightarrow z$$

At most k steps

$$A_{pr} \Rightarrow \dots \Rightarrow y$$

At most k steps

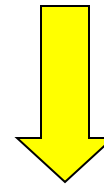


From induction hypothesis, in PDA:

$$(p, y, \lambda) \xrightarrow{*} (r, \lambda, \lambda)$$

$$A_{rq} \Rightarrow \dots \Rightarrow z$$

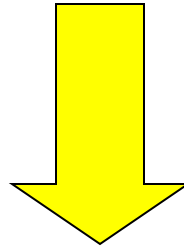
At most k steps



From induction hypothesis, in PDA:

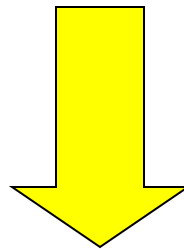
$$(r, z, \lambda) \xrightarrow{*} (q, \lambda, \lambda)$$

$$(p, y, \lambda)^* \succ (r, \lambda, \lambda) \quad (r, z, \lambda)^* \succ (q, \lambda, \lambda)$$



$$(p, yz, \lambda)^* \succ (r, z, \lambda)^* \succ (q, \lambda, \lambda)$$

since $w = yz$



$$(p, w, \lambda)^* \succ (q, \lambda, \lambda)$$

Type 3

Case 2: $A_{pq} \Rightarrow aA_{rs}b \Rightarrow \dots \Rightarrow w$

$k + 1$ steps

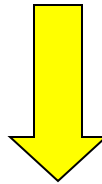
We can write $w = ayb$

$A_{rs} \Rightarrow \dots \Rightarrow y$

At most k steps

$$A_{rs} \Rightarrow \cdots \Rightarrow y$$

At most k steps

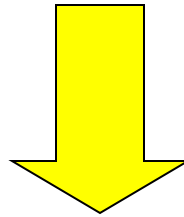


From induction hypothesis,
the PDA has computation:

$$(r, y, \lambda) \stackrel{*}{\succ} (s, \lambda, \lambda)$$

Type 3

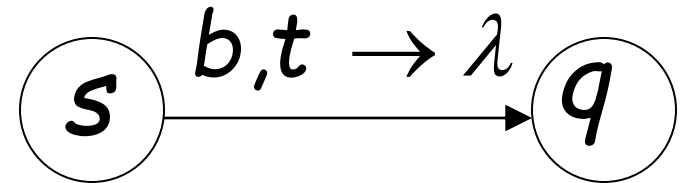
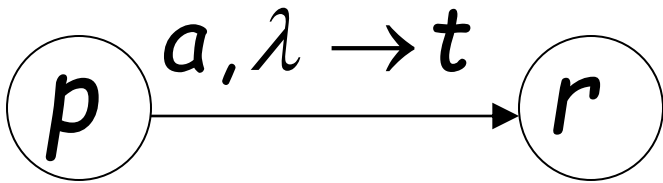
$$A_{pq} \Rightarrow aA_{rs}b \Rightarrow \dots \Rightarrow w$$

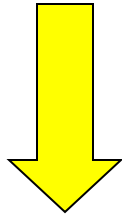
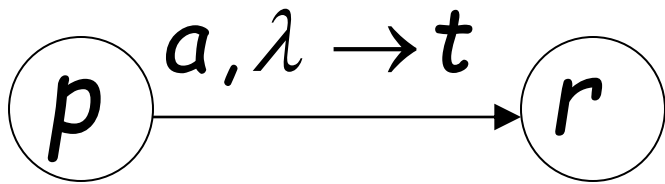


Grammar contains production

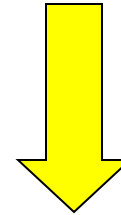
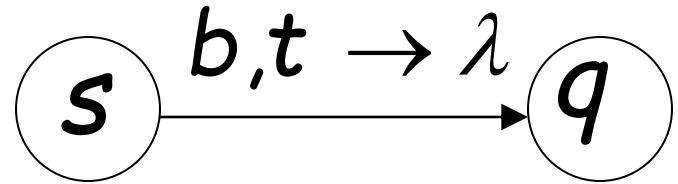
$$A_{pq} \rightarrow aA_{rs}b$$

And PDA Contains transitions





$$(p, ayb, \lambda) \succ (r, yb, t)$$



$$(s, b, t) \succ (q, \lambda, \lambda)$$

We know

$$(r, y, \lambda) \succ^* (s, \lambda, \lambda) \implies (r, yb, t) \succ^* (s, b, t)$$

We also know

$$(p, ayb, \lambda) \succ (r, yb, t)$$

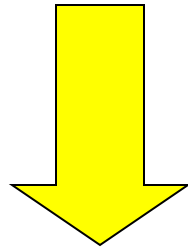
$$(s, b, t) \succ (q, \lambda, \lambda)$$

Therefore:

$$(p, ayb, \lambda) \succ (r, yb, t) \succ^* (s, b, t) \succ (q, \lambda, \lambda)$$

$$(p, ayb, \lambda) \succ (r, yb, t) \overset{*}{\succ} (s, b, t) \succ (q, \lambda, \lambda)$$

since $w = ayb$



$$(p, w, \lambda) \overset{*}{\succ} (q, \lambda, \lambda)$$

END OF PROOF

So far we have shown:

$$L(G) \subseteq L(M)$$

With a similar proof we can show

$$L(G) \supseteq L(M)$$

Therefore: $L(G) = L(M)$